

Hacia una Arquitectura de Software Auditable para la Composición Académica Agéntica: De la Metodología MARCO al Diseño de *MARCO-Platform*^{*}

*

*

Abstract: La integración de Modelos de Lenguaje de Gran Escala (LLM) en la producción científica exige conciliar la automatización agéntica con el control de autoría y las directrices éticas editoriales. La metodología MARCO (*Methodology of Agentic Reporting and Composition Orchestration*) propuso un flujo estructurado *Human-in-the-Loop* (HITL) cuya factibilidad operativa fue ilustrada mediante un ejercicio preliminar de auto-aplicación (*dogfooding*). No obstante, dicho caso de estudio expuso fragilidades estructurales compartidas por los pipelines agénticos actuales: vulnerabilidad ante bloqueos de scraping web (errores HTTP 429/503), deriva de extensión paramétrica no controlada (expansión del 67% fuera del flujo agéntico) y ausencia de telemetría objetiva para auditar la co-autoría humana. Este artículo responde a estas limitaciones presentando la **especificación de requisitos y la arquitectura de software de referencia de MARCO-Platform**: un banco de trabajo local de alta trazabilidad (*Local-First Auditable Workbench*) actualmente en fase de prototipado como Producto Mínimo Viable (MVP). La propuesta sintetiza cuatro patrones consolidados de ingeniería de software adaptados a la gobernanza editorial: (i) una máquina de estados finitos (FSM) con compuertas síncronas de supervisión humana; (ii) un esquema de memoria dual inspirado en *Event Sourcing* que desacopla la auditoría inmutable en disco de la ventana de contexto activa; (iii) un módulo de gobernanza asimétrica de tres niveles diseñado para focalizar la atención y reducir el sesgo de automatización; y (iv) un monitor paramétrico en tiempo real con políticas algorítmicas de compensación presupuestaria. Asimismo, se detalla la arquitectura de implementación del MVP en desarrollo y se formaliza una agenda de evaluación empírica orientada a la reproducibilidad y a la transparencia auditable de la ciencia co-producida con IA.

Keywords: sistemas multi-agente, Human-in-the-Loop, arquitectura de software, gobernanza editorial, composición académica asistida, trazabilidad de autoría, sesgo de automatización.

1. INTRODUCCIÓN

La integración de Modelos de Lenguaje de Gran Escala (LLM, por sus siglas en inglés) en la producción científica ha evolucionado rápidamente en los últimos tres años, polarizando el debate entre dos paradigmas divergentes: la automatización integral de la investigación sin supervisión humana directa (Ahn, 2024; Lu et al., 2024) y el uso conversacional ad-hoc y desarticulado (Silva et al., 2025). Iniciativas como *The AI Scientist* (Lu et al., 2024) o *CycleResearcher* (Weng et al., 2024) persiguen ciclos autónomos de descubrimiento y redacción de manuscritos. No obstante, la evidencia empírica acumulada advierte de forma consistente que la delegación agéntica no supervisada introduce patologías severas: incapacidad para discriminar sesgos metodológicos entre estudios contradictorios, propensión a alucinaciones referenciales y homogenización estilística del discurso científico (Khraisha et al., 2023; Scherbakov et al., 2024; Silva et al., 2025).

Frente a la automatización no supervisada, el marco metodológico MARCO (*Methodology of Agentic Reporting and Composition Orchestration*), propuesto en Mata-López (2026), postuló un enfoque *Human-in-the-Loop* (HITL) orientado a preservar el control conceptual y la dirección del investigador mediante fases secuenciales invariantes y reglas de gobernanza persistentes. Para ilustrar la viabilidad operativa preliminar del modelo, su autor documentó un ejercicio de auto-aplicación metodológica (*dogfooding* experimental) en el que MARCO fue empleado para estructurar y redactar su propio manuscrito fundacional. Si bien este ejercicio demostró que un flujo estructurado con agentes puede completar la redacción de un borrador técnico extenso (≈ 100 minutos de interacción agéntica y 18 artículos científicos procesados), el análisis riguroso de la experiencia pone de manifiesto que se trató de una prueba de concepto en etapa temprana (TRL 3-4), ejecutada sin grupo de control ni evaluadores ciegos independientes, dejando intactas importantes interrogantes sobre su robustez y escalabilidad.

Más relevante aún para la ingeniería de software es que dicho caso de estudio expuso un conjunto de fallas de

^{*} Investigación desarrollada en el marco del diseño de plataformas para orquestación agéntica auditable en publicaciones científicas.

infraestructura y de diseño sociotécnico recurrentes en los pipelines de generación de texto académico con LLMs: (i) la fragilidad de depender del scraping o APIs públicas no autenticadas, susceptibles a bloqueos por tasa de uso; (ii) la falta de mecanismos activos para contener la deriva de extensión (*length drift*), evidenciada cuando la curaduría humana desbordó en un 67% el límite editorial fijado; (iii) el riesgo inherente de fatiga cognitiva y sesgo de automatización (*automation bias*) ante la presentación indiferenciada de texto generado; y (iv) la ausencia de telemetría automatizada capaz de cuantificar con precisión la agencia y co-autoría humana para responder a las nuevas políticas editoriales de integridad científica (COPE, 2023; Elsevier, 2023).

Este artículo aborda estas problemáticas presentando la **especificación de requisitos y la arquitectura de software de referencia de MARCO-Platform**, un banco de trabajo de ejecución local (*Local-First Auditable Workbench*) actualmente en fase de desarrollo como Producto Mínimo Viable (MVP). La contribución no pretende postular nuevos algoritmos de inteligencia artificial ni proclamar superioridad empírica prematura sobre sistemas existentes. En cambio, su objetivo es: (1) formalizar el espacio de problemas y requisitos no funcionales (trazabilidad inmutable, tolerancia a fallos de red y control paramétrico estricto) inherentes a la co-autoría académica asistida; (2) sintetizar un diseño arquitectónico desacoplado, modular y tolerante a fallos mediante la adaptación de patrones consolidados de ingeniería de software (arquitectura hexagonal, máquinas de estados finitos, *Event Sourcing* y gobernanza asimétrica); (3) detallar el stack tecnológico y las consideraciones de despliegue del MVP en curso; y (4) establecer el protocolo experimental requerido para su posterior validación empírica con investigadores externos.

El resto del manuscrito se estructura de la siguiente manera: la Sección 2 analiza el estado del arte y examina la cobertura de requisitos en herramientas representativas; la Sección 3 analiza en detalle el espacio de problemas y las fracturas estructurales observadas en los flujos agénticos; la Sección 4 expone la arquitectura propuesta de cinco capas y sus contratos abstractos; la Sección 5 detalla la especificación del MVP, el dimensionamiento computacional analítico y las amenazas a la validez; y la Sección 6 formula las conclusiones y la agenda de validación empírica.

2. MARCO TEÓRICO Y ESTADO DEL ARTE

2.1 Del Prompting Conversacional a la Orquestación Agéntica

La adopción de modelos de lenguaje en la literatura científica ha transitado desde la asistencia conversacional no estructurada (Ahn, 2024) hacia sistemas agénticos modulares orientados a flujos de trabajo específicos. En el ámbito del cribado y síntesis de literatura, herramientas como LatteReview (Rouzrokh and Shariatnia, 2025), PROMPTHEUS (Torres et al., 2025) y los pipelines multi-agente de Sami et al. (2024) han demostrado reducciones apreciables en la carga operativa de las revisiones sistemáticas, mitigando el impacto del crecimiento exponencial de publicaciones (Dennstädt et al., 2024; Ye et al.,

2024). Paralelamente, frameworks generalistas como AutoGen (Wu et al., 2023) o sistemas de generación extensa basados en grafos como STORM (Shao et al., 2024) han sentado bases técnicas para coordinar múltiples agentes mediante diálogo sintético.

No obstante, los estudios empíricos sobre la calidad del texto generado por LLMs en contextos académicos señalan riesgos metodológicos persistentes: las alucinaciones de citas continúan manifestándose incluso en modelos avanzados (Scherbakov et al., 2024), la capacidad de los modelos para juzgar discrepancias de diseño metodológico entre estudios empíricos es limitada (Khraisha et al., 2023) y el estilo sintético homogéneo puede comprometer la originalidad discursiva (Silva et al., 2025). Aunque técnicas como el ajuste fino eficiente (PEFT) incrementan la precisión en la extracción de datos (Sušnjak et al., 2024), la literatura converge en que los sistemas híbridos que reservan al investigador humano la supervisión de decisiones críticas proporcionan el mejor balance entre aceleración del proceso y rigor epistemológico (Malik and Terzidis, 2025). Asimismo, infraestructuras pioneras de autoría colaborativa humano-IA (Pividori and Greene, 2023) han subrayado la necesidad de articular el proceso bajo esquemas de control de versiones y auditoría continua.

Bajo esta perspectiva, el paradigma *Human-in-the-Loop* (HITL) no debe entenderse como un estadio transitorio hacia la automatización plena, sino como una necesidad metodológica de primer orden para garantizar que el conocimiento generado responda a la intencionalidad, juicio y responsabilidad ética del investigador (Drori and Te'eni, 2024; Mata-López, 2026).

2.2 La Metodología MARCO: Principios y Caso de Estudio Inicial

La metodología MARCO propone organizar la producción de documentos académicos en cuatro fases secuenciales: Parametrización del destino editorial (Fase I), Curaduría literaria (Fase II), Redacción modular secuencial (Fase III) y Ensamblaje y revisión final (Fase IV), todo ello transversalizado por un repositorio de reglas persistentes (*rules*). La regla fundamental del flujo es la secuencialidad estricta: ninguna fase puede iniciarse sin la validación explícita del investigador sobre los artefactos de la fase anterior.

El caso de estudio fundacional (Mata-López, 2026) implementó este flujo de manera artesanal sobre un entorno local asistido por agentes en Antigravity IDE (Gemini 3.1 Pro y Claude Sonnet) y control de versiones sobre Windows 11. El autor reportó que las reglas persistentes lograron acotar las alucinaciones temáticas y que las intervenciones humanas se concentraron en decisiones terminológicas y estructurales. Sin embargo, al tratarse de un experimento de auto-observación sin réplica independiente ni evaluación ciega, sus resultados representan una prueba de viabilidad preliminar y un valioso registro de fricciones prácticas, mas no una validación empírica formal de rigor ni de generalizabilidad.

2.3 Brechas entre Prototipos de Investigación y Sistemas en Producción

La literatura en ingeniería de software para sistemas agénticos identifica que los prototipos de investigación frecuentemente colapsan ante condiciones reales de despliegue debido a cuatro carencias fundamentales (Kleppmann, 2017): falta de resiliencia frente a caídas de servicios externos, ausencia de persistencia transaccional del estado, acoplamiento rígido con librerías o APIs específicas y carencia de telemetría objetiva sobre la interacción usuario-sistema. Estas cuatro deficiencias se manifestaron nítidamente en la experiencia de MARCO y definen el conjunto de requisitos de ingeniería que MARCO-Platform aborda en su diseño.

2.4 Análisis de Requisitos y Estado del Arte

La arquitectura hexagonal o patrón de Puertos y Adaptadores (Cockburn, 2005) ofrece un principio idóneo para aislar el dominio metodológico de la volatilidad de los frameworks agénticos. Asimismo, el modelado mediante máquinas de estados finitos (FSM) con puntos de arbitraje síncronos (Rouzrokh and Shariatnia, 2025; Torres et al., 2025) y el uso de registros secuenciales inmutables (*append-only event logs*) (van der Aalst, 2016; Kleppmann, 2017) complementados con recuperación híbrida BM25 y vectorial (Robertson and Zaragoza, 2009; Johnson et al., 2021) constituyen la base técnica para diseñar un workbench de alta confiabilidad.

La Tabla 1 presenta un análisis comparativo de las capacidades reportadas en sistemas representativos del estado del arte frente al espacio de requisitos funcionales que orienta la especificación de diseño de MARCO-Platform.

Este análisis revela que los sistemas actuales se enfocan predominantemente en la fase de revisión bibliográfica o en la generación libre de contenido, existiendo un vacío arquitectónico respecto a herramientas locales integradas que combinen verificación referencial estricta, control de presupuesto de páginas y auditoría inmutable de la agencia humana.

3. ANÁLISIS DEL ESPACIO DE PROBLEMAS: FRAGILIDADES ESTRUCTURALES EN LA ORQUESTACIÓN AGÉNTICA

El análisis crítico del experimento fundacional de Mata-López (2026) y de la literatura reciente permite identificar cuatro problemáticas estructurales que condicionan el diseño de una plataforma robusta. Lejos de ser errores atribuibles exclusivamente a un caso aislado, constituyen fricciones inherentes a la intersección entre LLMs y flujos de publicación académica que deben resolverse a nivel arquitectónico.

3.1 La Paradoja de la Ingesta Bibliográfica

El scraping oportunista de repositorios públicos (arXiv, Semantic Scholar, Crossref) suele asumirse como el mecanismo natural de recolección bibliográfica agéntica. Sin embargo, en la práctica este enfoque colapsa debido a políticas legítimas de protección de infraestructura: límites

estrictos de tasa de llamadas (errores HTTP 429) y cortes temporales de servicio (HTTP 503). Esta tensión entre la autonomía pretendida por los agentes y las defensas técnicas de los servidores académicos compromete la reproducibilidad del pipeline.

Una plataforma académica en producción no puede depender de accesos informales. MARCO-Platform resuelve esta contradicción mediante una política de desacoplamiento: sustituye el scraping web en caliente por un pipeline de ingesta local que opera sobre repositorios de PDFs previamente curados por el investigador mediante gestores estándar (Zotero, Mendeley, BibTeX), complementado opcionalmente con conectores autenticados vía API keys oficiales. Si bien esto delega inicialmente la curaduría del corpus al investigador, garantiza la estabilidad del sistema, elimina bloqueos de red y permite una ejecución 100% reproducible y desconectada (*local-first*).

3.2 La Deriva de Extensión Paramétrica (Length Drift)

Los LLMs exhiben dificultades notorias para adherirse a restricciones duras de longitud en interacciones complejas o de múltiples turnos. En el caso de estudio de MARCO, la plantilla editorial imponía un límite de 6 páginas en doble columna; no obstante, el texto final alcanzó 10 páginas (+67%). Aunque este exceso fue atribuido a una decisión deliberada del autor durante la revisión humana, desde la perspectiva de sistemas evidencia que la restricción operó como una instrucción estática en el prompt, sin un mecanismo activo que supervisara la longitud acumulada a lo largo de las ediciones.

Para prevenir la deriva paramétrica, MARCO-Platform especifica un monitor de presupuesto en tiempo real que mide continuamente el conteo de palabras por sección tanto durante la generación agéntica como durante la intervención del investigador, proporcionando mecanismos de alerta temprana y políticas algorítmicas de condensación o redistribución de espacio.

3.3 Sobrecarga Cognitiva y Sesgo de Automatización (Automation Bias)

La premisa de un investigador humano que valida críticamente cada borrador generado por un LLM choca frontalmente con la fatiga cognitiva: la exposición continuada a bloques extensos de texto sintético induce una aceptación pasiva o acrítica (*rubber-stamping*), neutralizando la eficacia del esquema HITL (Parasuraman and Manzey, 2010; Drori and Te'eni, 2024).

Presentar borradores completos de sección sin estratificación de riesgos favorece el sesgo de automatización. La arquitectura de MARCO-Platform aborda esta limitación mediante un modelo de gobernanza asimétrica que segmenta la supervisión en tres niveles: automatiza las correcciones de bajo impacto sintáctico y concentra la atención del investigador en las afirmaciones de alta incertidumbre referencial o en los hitos transicionales mayores.

3.4 Déficit de Telemetría Automatizada de Auditoría

En el experimento de MARCO (Mata-López, 2026), los datos sobre tiempo dedicado e intervenciones humanas se

Table 1. Matriz de requisitos funcionales para plataformas de co-autoría agéntica: capacidades evaluadas en sistemas existentes vs. especificación de diseño de MARCO-Platform (MVP en desarrollo).

Dimensión de Requisito	LatteReview / Sami et al.	PROMPTHEUS	AutoGen / STORM	MARCO (2026)	MARCO-Platform (MVP)
Orquestación agéntica multi-fase	Parcial	Parcial	Sí	Sí	Sí (FSM modular)
Supervisión HITL estructurada	No	No	No	Sí (Artesanal)	Sí (Compuertas FSM)
Auditoría inmutable de co-autoría	No	No	No	No	Sí (Stream A JSONL)
Control paramétrico de extensión	No	No	No	No	Sí (Monitor activo)
Ingesta local de PDFs (sin red)	No	No	No	No	Sí (Grobid/OCR)
Verificación referencial RAG local	No	Parcial	No	No	Sí (Híbrido RRF)

obtuvieron mediante anotaciones cualitativas retrospectivas. Esta carencia de telemetría objetiva impide evaluar cuantitativamente la reproducibilidad del proceso y dificulta el cumplimiento riguroso de las directrices éticas editoriales sobre el uso de IA (COPE, 2023; Elsevier, 2023). La declaración binaria tradicional (¿se utilizó IA: sí/no?) resulta insuficiente para reflejar la co-creación; se requiere un registro granular e inmutable que trace las contribuciones humanas y sintéticas a nivel de frase y sección.

La Tabla 2 sintetiza estas cuatro problemáticas estructurales y las soluciones arquitectónicas especificadas en MARCO-Platform.

4. PROPUESTA DE ARQUITECTURA: MARCO-PLATFORM

MARCO-Platform se concibe como un banco de trabajo local de alta trazabilidad (*Local-First Auditable Workbench*) estructurado bajo el patrón de arquitectura hexagonal (Cockburn, 2005) y dirigido por eventos. El principio rector es que el dominio metodológico de la autoría académica debe aislarse de la volatilidad tecnológica de los frameworks de agentes mediante contratos abstractos. La Fig. 1 detalla la arquitectura de cinco capas del sistema.

4.1 Capa 2: Núcleo de Orquestación — Contratos Hexagonales

El núcleo de orquestación implementa una máquina de estados finitos (FSM) que hace cumplir la secuencia invariante de fases mediante cuatro contratos de interfaz:

HITLBarrier: Suspende la ejecución agéntica al término de cada fase o sección mayor, notificando a la interfaz de usuario y bloqueando el avance hasta recibir una confirmación explícita del investigador registrada en el log de eventos.

GovernanceInterceptor: Filtro perimetral que intercepta cada salida del modelo antes de exponerla al usuario, canalizándola a través de: (a) verificación referencial frente al corpus local (*Veracity Guard*); (b) análisis de estilo para suprimir muletillas generativas; y (c) control de conformidad paramétrica seccional.

TrajectoryLogger: Escribe de manera asíncrona en el *Stream A* de auditoría (Sección 4.4) cada evento del

sistema (prompts emitidos, respuestas crudas, latencias, tokens y diferencias de edición humana), garantizando un registro inmutable sin degradar la respuesta interactiva.

SectionSkillRunner: Encapsula la lógica de redacción de cada sección temática mediante la interfaz `run(context: SectionContext) → SectionDraft`. Desacopla la lógica disciplinar de la sección del motor de ejecución y de las reglas generales de gobernanza.

Este desacoplamiento minimiza el impacto de los frecuentes *breaking changes* en librerías agénticas (p. ej., LangGraph, AutoGen) o de eventuales discrepancias en la semántica de serialización asíncrona (*leaky abstractions*), confinando las adaptaciones a la periferia de la arquitectura.

4.2 Capa 4: Ingestión Paramétrica y Extracción Local

Parser de Plantillas: Analiza plantillas `.docx` o `.tex` para extraer un esquema JSON estructurado con las restricciones del evento editorial:

```
{
  "max_pages": 6,
  "column_layout": "double",
  "font": "Times New Roman 10pt",
  "citation_style": "APA",
  "abstract_max_words": 150,
  "mandatory_sections": ["Abstract",
    "Introduction", "Methodology",
    "Results", "Conclusions", "References"],
  "figure_caption": "centered_top",
  "table_caption": "centered_top"
}
```

Este esquema se inyecta como metadato activo en los encabezados YAML de los archivos de sección para guiar al monitor de presupuesto.

Pipeline de Ingesta Local de Documentos: Opera sobre PDFs locales pre-curados. Incorpora una arquitectura de doble parser: **Grobid** (v0.8+) como motor primario optimizado para CPU sobre PDFs nativos digitales, extrayendo metadatos y secciones en formato estructurado; ante bajas puntuaciones de confianza (< 0,7) o páginas escaneadas con tipografía matemática compleja, el sistema deriva el archivo a **Nougat** en estaciones con GPU (VRAM ≥ 8 GB), o bien a un pipeline ligero de OCR de-

Table 2. Espacio de problemas en la composición agéntica y soluciones arquitectónicas especificadas en MARCO-Platform.

#	Problemática observada	Manifestación en la práctica	Solución en MARCO-Platform
1	Paradoja de la ingesta	Bloqueos HTTP 429/503 por scraping de APIs públicas	Conectores autenticados + ingesta local de PDFs pre-curados
2	Deriva de extensión (<i>length drift</i>)	Desbordamiento no monitoreado del límite de páginas	Monitor paramétrico activo con balance y condensación
3	Sesgo de automatización HITL	Aceptación acrítica de borradores por fatiga cognitiva	Gobernanza asimétrica de 3 niveles con atención focalizada
4	Déficit de telemetría de auditoría	Registros cualitativos imprecisos de agencia humana	Log inmutable <i>append-only</i> (JSONL) con métricas de diff

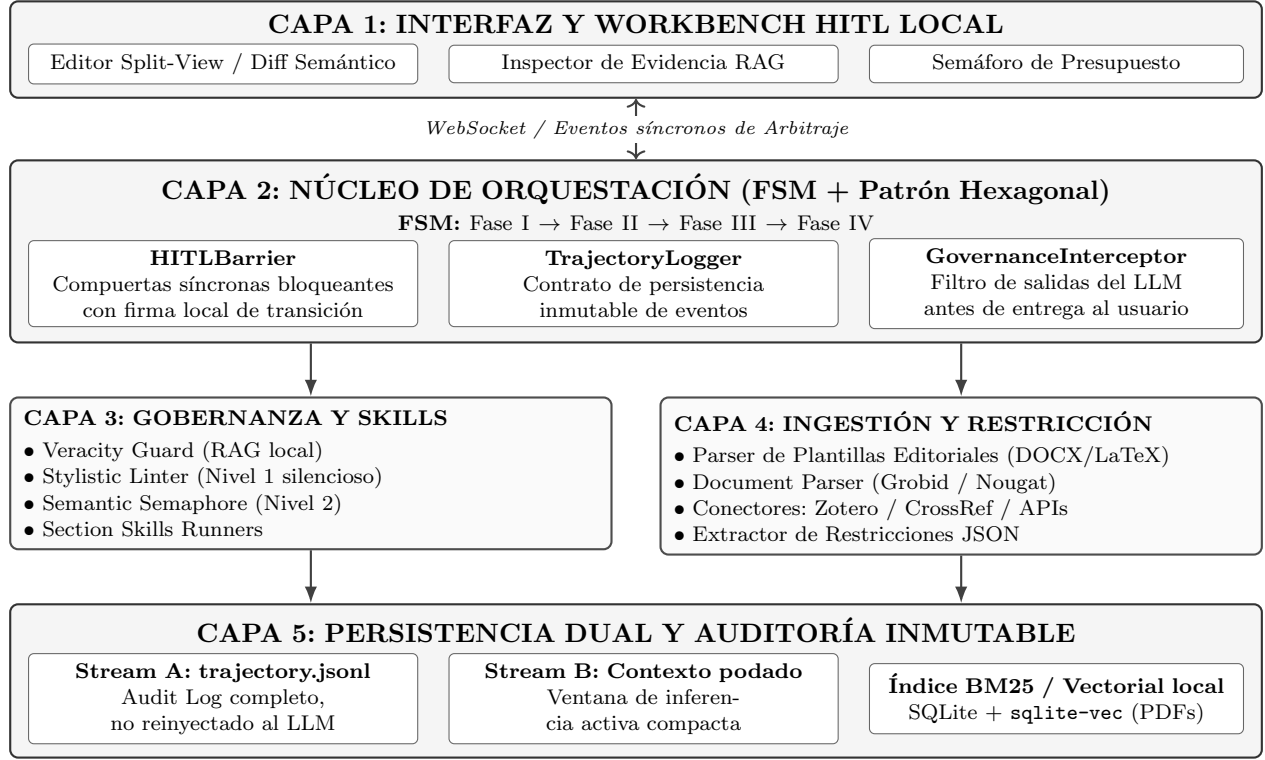


Fig. 1. Arquitectura de cinco capas de MARCO-Platform. Las capas 3 y 4 interactúan con el núcleo de orquestación (Capa 2) mediante contratos abstractos desacoplados del framework subyacente.

terminista (PyMuPDF + Tesseract) en entornos basados exclusivamente en CPU.

Recuperación Híbrida RRF: La búsqueda de fragmentos combina un índice léxico **BM25** (rank_bm25, para concordancia exacta de autores o términos técnicos) (Robertson and Zaragoza, 2009) y un índice **vectorial** (sqlite-vec con embeddings all-MiniLM-L6-v2, para concordancia conceptual) (Johnson et al., 2021). Los rankings resultantes se fusionan mediante *Reciprocal Rank Fusion* (RRF), equilibrando precisión y cobertura semántica sin requerir ajuste fino por disciplina.

4.3 Capa 3: Gobernanza HITL Asimétrica de Tres Niveles

Para contrarrestar la fatiga cognitiva y el sesgo de automatización (Parasuraman and Manzey, 2010; Drori and Te'eni, 2024), la intervención humana se organiza en tres niveles jerárquicos:

Nivel 1 — Linters Estáticos Silenciosos: Filtros deterministas y rápidos que operan sin intervención del usuario. Suprimen muletillas generativas autorreferen-

ciales (¡¡cabe señalar!!), ¡¡es importante destacar!!), estandarizan unidades de medida y verifican encabezados. Cada corrección se anota silenciosamente en el log.

Nivel 2 — Semáforo Semántico de Citas: Contrasta vectorialmente cada afirmación que contenga una cita bibliográfica frente a los fragmentos del corpus indexado:

- **Verde** (similitud $\geq 0,82$): Concordancia textual y semántica robusta. Aprobación rápida con un clic.
- **Amarillo** (similitud $\in [0,55; 0,81]$): Concordancia parcial o ambigua; despliega la fuente en el Inspector de Evidencia.
- **Rojo** (similitud $< 0,55$ o fuente ausente): Riesgo de alucinación referencial; requiere decisión explícita.

Estos umbrales son valores heurísticos nominales de diseño parametrizables en `governance.yaml`, cuya calibración empírica forma parte del plan de evaluación del MVP.

Nivel 3 — Compuertas Transicionales Troncales: Bloqueos síncronos de la FSM al término de cada fase o sección. La transición registra en el log un evento `PHASE_TRANSITION` que almacena: el hash SHA-

256 del artefacto aprobado, el identificador del usuario (`os.getLogin()`), marca temporal ISO 8601 y un comentario del investigador. Se subraya que este mecanismo opera como un **sello de integridad direccionado por contenido** (análogo a un commit en Git) para control de versiones y auditoría de buena fe, no como una firma de no-repudio criptográfico ante actores adversarios.

La Tabla 3 sintetiza los tres niveles de gobernanza.

4.4 Capa 5: Memoria Dual inspirada en Event Sourcing

Para evitar la degradación del modelo y el incremento descontrolado de costos por explosión de contexto (*context explosion*) (Kleppmann, 2017), la plataforma desacopla la memoria en dos corrientes:

Stream A (Auditoría Inmutable): Log secuencial *append-only* (`trajectory.jsonl`) gestionado por el contrato `Trajectory-Logger`. Registra prompts, respuestas crudas, latencias, tokens y diferencias de edición humana. **No se reinyecta al contexto del LLM** en fases posteriores; se reserva para reproducibilidad y generación del reporte de transparencia.

Stream B (Ventana de Inferencia Podada): Cada *skill* de sección opera como una función pura sobre una ventana compacta: recibe únicamente el artefacto consolidado de la fase previa (matrices temáticas), las restricciones paramétricas y las reglas de gobernanza. Las citas necesarias se recuperan bajo demanda mediante consultas RAG al índice local, manteniendo un costo computacional predecible.

4.5 Capa 1: Workbench HITL Local

La interfaz de usuario implementa una disposición de panel dividido:

Panel Izquierdo (Editor de Sección): Presenta el borrador generado con las correcciones del Nivel 1 aplicadas y las alertas del Nivel 2 señalizadas. El editor registra las modificaciones del investigador mediante dos métricas complementarias: el **diff léxico** (distancia Levenshtein entre tokens) y el **diff semántico** (distancia coseno entre embeddings del borrador original y el editado). Se advierte una limitación epistemológica: la distancia coseno es un indicador macro-semántico que no detecta necesariamente inversiones de significado lógico (p. ej., la alteración de una negación), por lo que ambas métricas se almacenan de forma independiente para evitar lecturas simplistas sobre la agencia humana.

Panel Derecho (Inspector de Evidencia): Muestra el fragmento original del PDF que respalda la cita seleccionada, junto con su puntuación de similitud.

Barra de Presupuesto Paramétrico: Monitorea el consumo de palabras frente al cupo asignado. Ante un desbordamiento durante la curaduría humana, ofrece dos opciones algorítmicas asistidas: **condensación automática** (reducción agéntica preservando invariantes técnicas y citas) o **redistribución de presupuesto** (reasignación proporcional del cupo residual entre secciones no finalizadas).

4.6 Generación Automática del Reporte de Transparencia de Autoría

Al finalizar el ensamblaje (Fase IV), el sistema procesa el archivo `trajectory.jsonl` para emitir un reporte cuantitativo que desglosa: (a) distribución de tokens generados vs. editados manualmente; (b) índice de intervención editorial (distancia de edición promedio); (c) tiempo de revisión humana acumulado por fase; y (d) trazabilidad referencial por nivel de confianza. Este reporte proporciona una base objetiva para fundamentar las declaraciones de co-autoría exigidas por comités editoriales como COPE (2023) y políticas editoriales como las de Elsevier (2023) y Nature (Mata-López, 2026).

5. ESPECIFICACIÓN DEL MVP, ANÁLISIS DE FACTIBILIDAD Y LIMITACIONES DE DISEÑO

5.1 Dimensionamiento Computacional y Parámetros de Diseño para el MVP

A partir de la formalización del flujo de trabajo y del modelado analítico proyectado desde las trazas del caso de estudio preliminar, se derivan los siguientes parámetros de dimensionamiento computacional para un artículo estándar de 6 páginas (≈ 3.000 palabras) sobre un corpus de 18 documentos:

- **Fase II (Extracción estructurada):** Demanda estimada de 45.000 a 60.000 tokens de entrada, con latencias de indexación de 8 a 15 minutos en procesamiento por lotes asíncrono.
- **Fase III (Redacción por sección con RAG):** Estimación de 8.000 a 12.000 tokens por sección y 90 a 120 segundos de procesamiento en segundo plano.
- **Overhead de Gobernanza:** Incremento proyectado del 15–20% en tokens por llamada (Linters y Semáforo) respecto al prompting directo, situando el overhead acumulado del pipeline completo entre $2,2\times$ y $2,8\times$.

Es fundamental enfatizar que estas cifras representan **parámetros analíticos de diseño y dimensionamiento presupuestario** para la implementación del MVP, y no métricas de benchmark de software en producción. Su contrastación empírica formal constituye uno de los objetivos centrales del plan de pruebas del prototipo.

5.2 Justificación del Overhead Computacional

El incremento en consumo computacional se justifica frente a las severas repercusiones académicas de las alucinaciones referenciales y los errores factuales en literatura revisada por pares (Scherbakov et al., 2024; Silva et al., 2025). En la producción científica, la prioridad de optimización no es la inmediatez de respuesta en milisegundos, sino la integridad documental y la verificabilidad de las fuentes citadas.

5.3 Generalización a Formatos Editoriales

El Parser de Plantillas distingue entre formatos declarativos estructurados (`.docx` mediante OpenXML o esquemas YAML), donde la extracción de márgenes, extensiones

Table 3. Niveles de gobernanza HITL asimétrica, desencadenantes y trazabilidad en MARCO-Platform.

Nivel	Denominación	Desencadenante	Acción del Sistema	Intervención Humana	Registro en Log
1	Lintor silencioso	Salida cruda del LLM	Corrección automática determinista	Ninguna	Sí: tipo de corrección y texto original
2	Semáforo semántico	Afirmación con cita	Clasificación verde/amarillo/rojo	Solo en amarillo y rojo	Sí: similitud y decisión tomada
3	Compuerta transicional	Fin de fase macro	Suspensión de la FSM; pantalla de estado	Obligatoria: firma de transición	Sí: timestamp, hash SHA-256, identidad

y fuentes es altamente determinista, y documentos basados en código fuente (`.tex`), donde la ambigüedad de macros personalizadas requiere combinar reglas heurísticas sobre comandos estándar con la posibilidad de parametrización manual asistida en `governance.yaml`.

5.4 Mitigación de la Fatiga de Decisión

Al automatizar correcciones menores en el Nivel 1 y concentrar la interacción en las discrepancias referenciales del Nivel 2 y los hitos del Nivel 3, el sistema reduce la fricción operativa y previene la saturación cognitiva del investigador, preservando su juicio crítico para las decisiones de mayor relevancia conceptual.

5.5 Arquitectura Técnica y Estado de Implementación del MVP

La Tabla 4 detalla el stack tecnológico seleccionado para la construcción del Producto Mínimo Viable (MVP) de MARCO-Platform, priorizando la ejecución local y componentes de código abierto con interfaces estables.

El prototipado del MVP se orienta a ejecutarse en estaciones de trabajo estándar (16 GB RAM, CPU multinúcleo), utilizando llamadas a API o inferencia local ligera vía Ollama para las tareas de filtrado, y ejecutando Grobid sobre CPU. En equipos sin aceleración por GPU, el fallback visual de Nougat se reemplaza por OCR clásico determinista para mantener latencias operativas viables.

5.6 Amenazas a la Validez y Limitaciones del Estado Actual

En concordancia con los estándares de rigor en ingeniería de software, se explicitan las siguientes amenazas a la validez:

- **Amenaza a la Validez de Constructo:** Las métricas propuestas para evaluar la co-autoría (distancia Levenshtein y distancia coseno) son indicadores *proxy* cuantitativos. Aunque capturan la magnitud del cambio textual y la divergencia vectorial, no reflejan de manera perfecta la profundidad del juicio conceptual humano. La calibración de estas métricas contra evaluaciones ciegas de revisores humanos forma parte de la agenda empírica del MVP.
- **Amenaza a la Validez Externa:** La especificación se fundamenta en flujos estructurados de ingeniería y ciencias computacionales. Su generalización a disciplinas humanísticas o hermenéuticas, con estructuras

argumentales no convencionales, requiere adaptar las reglas de gobernanza y flexibilizar los modelos de FSM.

- **Estado de la Validación Empírica:** Se subraya con total transparencia académica que este artículo presenta la **especificación de requisitos y la arquitectura de referencia del sistema**, acompañada de la especificación técnica de su MVP. No se reportan datos de despliegue en producción ni estudios de usuario finalizados; la validación cuantitativa y cualitativa se iniciará una vez concluida la integración del prototipo.

6. CONCLUSIONES Y AGENDA DE VALIDACIÓN EMPÍRICA

Este artículo ha presentado la especificación formal de requisitos y la arquitectura de software de referencia de **MARCO-Platform**, un banco de trabajo local de alta trazabilidad diseñado para transformar la orquestación agéntica artesanal en una infraestructura formal, resiliente y auditable.

Las aportaciones principales del trabajo residen en: (1) el análisis sistemático del espacio de problemas y las fragilidades observadas en flujos agénticos de redacción académica (bloqueos de ingesta, deriva paramétrica, sesgo de automatización y déficit de telemetría); (2) la síntesis de un diseño modular de cinco capas basado en patrones consolidados de ingeniería de software (arquitectura hexagonal, máquinas de estados finitos, memoria dual tipo *Event Sourcing* y gobernanza asimétrica jerarquizada); (3) la especificación técnica detallada para la construcción de su MVP; y (4) la formalización de un marco de amenazas a la validez y directrices de auditoría alineadas con los estándares éticos editoriales contemporáneos.

La agenda de trabajo futuro se articula en tres fases inmediatas:

- (1) **Finalización y empaquetado del MVP:** Culminar la integración de los componentes especificados en la Tabla 4 (LangGraph, `sqlite-vec`, Grobid y Streamlit) en un repositorio abierto reproducible.
- (2) **Benchmark controlado de robustez:** Evaluar formalmente el pipeline sobre un corpus diverso de artículos científicos, cuantificando la reducción de alucinaciones mediante el Semáforo Semántico, la precisión de extracción de los parsers y el consumo real de tokens y latencias frente a las proyecciones analíticas.
- (3) **Estudio de usuario con investigadores:** Realizar pruebas empíricas con paneles de investigadores inde-

Table 4. Stack tecnológico seleccionado para la implementación del MVP de MARCO-Platform.

Capa arquitectónica	Componente del MVP	Alternativa viable
Orquestación FSM	LangGraph 0.2+ (Python)	Temporal.io (Go/Python)
Inferencia LLM (API remota)	Claude Sonnet / Gemini Pro	—
Inferencia LLM (Local)	Ollama + Llama 3.1 8B	LM Studio
Parser de PDFs primario	Grobid 0.8 (born-digital, CPU)	CERMINE
Parser de PDFs fallback	Nougat 0.1 (GPU) / PyMuPDF + Tesseract (CPU)	MathPix
Índice léxico BM25	<code>rank_bm25</code> (Python)	Elasticsearch local
Índice vectorial local	<code>sqlite-vec</code> + <code>sentence-transformers</code>	ChromaDB local
Fusión de búsqueda	Reciprocal Rank Fusion (RRF, algoritmo propio)	—
Interfaz de usuario HITL	Streamlit 1.3+	Gradio / Electron + Monaco
Log de auditoría	JSONL <i>append-only</i> en disco local	—
Firma de transición HITL	Hash SHA-256 nativo (<code>hashlib</code>)	—
Integración bibliográfica	Exportación RIS/BibTeX desde Zotero	Mendeley

pendientes para evaluar la usabilidad del Workbench, medir la atenuación del sesgo de automatización y comparar el cumplimiento de presupuestos editoriales frente a grupos de control que empleen prompting directo.

La orquestación de modelos de lenguaje en la redacción académica no debe comprometer la integridad científica. Al situar la supervisión humana fundamentada y la auditoría inmutable en el centro de su arquitectura, MARCO-Platform sienta las bases de ingeniería para que la co-creación con inteligencia artificial constituya un proceso metodológicamente riguroso, éticamente defendible y plenamente acreditable ante la comunidad científica.

ACKNOWLEDGEMENTS

Los autores agradecen el soporte del cuerpo académico y de los revisores del congreso CITCA 2026. Los autores declaran no tener conflictos de intereses que puedan haber influido en el diseño o presentación de este trabajo.

DECLARACIÓN DE USO DE INTELIGENCIA ARTIFICIAL

En concordancia con los principios de transparencia defendidos en este artículo, se declara que modelos de lenguaje de gran escala fueron utilizados como asistentes en tareas de soporte documental, cotejo de bibliografía y depuración de estilo bajo supervisión y dirección del autor. El diseño arquitectónico, el análisis crítico, las decisiones de modelado de software y las conclusiones finales son de responsabilidad intelectual exclusiva de los autores.

REFERENCES

- Ahn, S. (2024). The transformative impact of large language models on medical writing and publishing: current applications, challenges and future directions. *The Korean Journal of Physiology & Pharmacology*, 28(5), 393–401.
- Cockburn, A. (2005). Hexagonal architecture (Ports and Adapters Pattern). Alistair Cockburn Design Patterns.
- COPE (2023). Authorship and AI tools: COPE position statement. Committee on Publication Ethics.
- Dennstädt, F., Zink, J., Putora, P., Hastings, J., and Cihoric, N. (2024). Title and abstract screening for literature reviews using large language models: an exploratory study in the biomedical domain. *Systematic Reviews*, 13, 2575.
- Drori, I. and Te’eni, D. (2024). Human-in-the-loop AI reviewing: Feasibility, opportunities, and risks. *Journal of the Association for Information Systems*, 25(7).
- Elsevier (2023). The use of AI and AI-assisted technologies in writing for Elsevier. Elsevier Publishing Guidelines.
- Johnson, J., Douze, M., and Jégou, H. (2021). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535–547.
- Khraisha, Q., Put, S., Kappenberg, J., Warraitch, A., and Hadfield, K. (2023). Can large language models replace humans in systematic reviews? Evaluating GPT-4’s efficacy in screening and extracting data from peer-reviewed and grey literature in multiple languages. *Research Synthesis Methods*, 15(4), 616–626.
- Kleppmann, M. (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O’Reilly Media.
- Lu, C., Lu, C., Lange, R., Foerster, J., Clune, J., and Ha, D. (2024). The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*.
- Malik, F. and Terzidis, O. (2025). A hybrid framework for creating artificial intelligence-augmented systematic literature reviews. *Management Review Quarterly*.
- Mata-López, W.A. (2026). MARCO: Methodology of agentic reporting and composition orchestration for academic documents. In *Memorias del Congreso Internacional de Tendencias en Computación Aplicada (CITCA 2026)*. Universidad de Colima, Coquimatlán, Colima, México.
- Parasuraman, R. and Manzey, D.H. (2010). Complacency and bias in human use of automation: An attentional integration. *Human Factors*, 52(3), 381–410.
- Pivideri, M. and Greene, C.S. (2023). A publishing infrastructure for AI-assisted academic authoring. *bioRxiv preprint bioRxiv:2023.01.21.525030*.
- Robertson, S. and Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.
- Rouzrokh, P. and Shariatnia, M. (2025). LatteReview: A multi-agent framework for systematic review automation using large language models. *arXiv preprint arXiv:2501.05468*.
- Sami, M., Rasheed, Z., Kemell, K., Waseem, M., Kilamo, T., Saari, M., Nguyen-Duc, A., Systä, K., and Abrahamsson, P. (2024). System for systematic literature review using multiple AI agents: Concept and an empirical evaluation. *arXiv preprint arXiv:2403.08399*.

- Scherbakov, D., Hubig, N., Jansari, V., Bakumenko, A., and Lenert, L. (2024). The emergence of large language models as tools in literature reviews: a large language model-assisted systematic review. *Journal of the American Medical Informatics Association*, 32, 1071–1086.
- Shao, Y., Jiang, Y., Kanell, T.A., Xu, P., Khattab, O., and Lam, M.S. (2024). Assisting in writing wikipedia-like articles from scratch with large language models. *arXiv preprint arXiv:2402.14207*.
- Silva, J., Gouveia, R., Zielinski, K., Oliveira, M., Amancio, D., Bruno, O., and Oliveira, O. (2025). AI-assisted tools for scientific review writing: Opportunities and cautions. *ACS Applied Materials & Interfaces*, 17, 47795–47805.
- Sušnjak, T., Hwang, P., Reyes, N., Barczak, A., McIntosh, T., and Ranathunga, S. (2024). Automating research synthesis with domain-specific large language model fine-tuning. *ACM Transactions on Knowledge Discovery from Data*, 19, 1–39.
- Torres, J., Mulligan, C., Jorge, J., and Moreira, C. (2025). PROMPTHEUS: A human-centered pipeline to streamline systematic literature reviews with large language models. *Information*, 16(5), 420.
- van der Aalst, W.M. (2016). *Process Mining: Data Science in Action*. Springer Berlin Heidelberg.
- Weng, Y., Zhu, M., Bao, G., Zhang, H., Wang, J., Zhang, Y., and Yang, L. (2024). CycleResearcher: Improving automated research via automated review. *arXiv preprint arXiv:2411.00816*.
- Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A.H., White, R., Burger, D., and Wang, C. (2023). AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*.
- Ye, A., Maiti, A., Schmidt, M., and Pedersen, S. (2024). A hybrid semi-automated workflow for systematic and literature review processes with large language model analysis. *Future Internet*, 16(5), 167.